

HR Management System

End-to-End DevOps Deployment Guide (AWS / EKS)

Stack: Java Spring Boot (Backend) + React (Frontend) + PostgreSQL (RDS) + Docker + Kubernetes (EKS) + Jenkins

Prepared for: DevOps Internship — SaiTeja Infotech

Cloud Platform: Amazon Web Services (AWS)

Table of Contents

Table of Contents	2
1. Architecture Overview.....	4
1.1 High-Level Flow	4
1.2 Design Principles Applied	4
2. AWS Account & IAM Setup.....	5
2.1 Root Account Hardening	5
2.2 IAM User for Daily Operations	5
2.3 Billing & Cost Controls	5
2.4 CLI Configuration	5
△ Common Issues — Account Setup	5
3. Infrastructure as Code — Terraform	6
3.1 main.tf — Provider & Backend	6
3.2 vpc.tf — VPC, Subnets, NAT (Multi-AZ)	6
3.3 eks.tf — EKS Cluster (multi-AZ worker nodes)	7
3.4 ecr.tf — Container Registries	7
3.5 rds.tf — PostgreSQL (isolated DB subnet)	7
3.6 iam.tf — IRSA Role for Backend (Secrets Manager access).....	8
3.7 Apply.....	8
△ Common Issues — Terraform / Infra	8
4. Containerization	10
4.1 Backend Dockerfile (Spring Boot).....	10
4.2 Frontend Dockerfile (React)	10
4.3 Build, Scan & Push (manual reference — Jenkins automates this).....	10
△ Common Issues — Containerization	10
5. Jenkins CI/CD Pipeline	11
5.1 Jenkins Master on EKS	11
5.2 Jenkinsfile — Full Pipeline	11
△ Common Issues — Jenkins / CI-CD	12
6. Kubernetes Deployment Manifests.....	13
6.1 Backend Deployment + Service	13
6.2 Frontend Deployment + Service	13
6.3 HPA — Autoscaling for Both Services.....	14
6.4 Nginx Ingress — Path-Based Routing	14
△ Common Issues — Kubernetes / Ingress	14
7. Secrets Management — Secrets Manager + IRSA + ESO.....	16

7.1 Store Secrets in AWS Secrets Manager	16
7.2 Install External Secrets Operator (ESO)	16
7.3 SecretStore + ExternalSecret (backend only)	16
7.4 Non-Sensitive Config — ConfigMap (frontend + backend)	16
⚠ Common Issues — Secrets	17
8. Monitoring — Prometheus, CloudWatch, Grafana	18
8.1 Expose Spring Boot Metrics	18
8.2 Amazon Managed Service for Prometheus (AMP)	18
8.3 AWS Managed Grafana	18
8.4 Alerts	18
⚠ Common Issues — Monitoring	18
9. Security & Compliance Checklist	20
10. Cost Optimization	21
11. Deployment Checklist	22

1. Architecture Overview

This deployment follows a multi-AZ, production-style AWS architecture: source control triggers a Jenkins CI/CD pipeline, images are scanned and pushed to ECR, deployment happens on EKS via Helm with secrets injected through IRSA + External Secrets Operator, traffic is routed via Route 53 → ALB → Nginx Ingress, and the system is observed via Amazon Managed Prometheus and Managed Grafana.

1.1 High-Level Flow

```
Git push → Jenkins CI (build + test + SonarQube) → Docker build + Trivy scan → ECR →  
Jenkins CD (Helm) → IRSA + ESO secrets → Rolling update on EKS (multi-AZ) →  
Route53 → ALB → Nginx Ingress → Pods (HPA) → RDS (backend only) →  
Backend → Microsoft Graph API (Teams/Outlook) → AMP + Grafana monitoring → Alerts
```

1.2 Design Principles Applied

- Regional AWS services (Route 53, ECR, Secrets Manager, AMP, Managed Grafana) are NOT tied to any single AZ or subnet.
- EKS worker nodes and the ALB are spread across all 3 AZs for high availability.
- Secrets (DB credentials, JWT keys, Graph API tokens) are injected only into backend pods — never the frontend.
- Non-sensitive configuration goes through Kubernetes ConfigMaps, not Secrets Manager.
- Root AWS account is never used for daily operations — a dedicated IAM user with MFA handles all provisioning.

2. AWS Account & IAM Setup

2.1 Root Account Hardening

- Enable MFA on the root account immediately.
- Do not generate access keys for the root user — delete any that exist.
- Use root only for billing changes, account closure, or IAM recovery.

2.2 IAM User for Daily Operations

```
IAM → Users → Create User
Username: haru-devops
Console access: enabled
Permissions: AdministratorAccess (managed policy)
MFA: enabled (authenticator app)
```

2.3 Billing & Cost Controls

- Billing alerts enabled, notifications sent to primary email.
- CloudWatch billing alarm configured at multiple thresholds (e.g. \$5 / \$15 / \$30).
- AWS Budgets: monthly cost budget with 80% and 100% alert thresholds.
- CloudTrail enabled across all regions for audit logging.

2.4 CLI Configuration

```
aws configure
# AWS Access Key ID: <from haru-devops IAM user, NOT root>
# AWS Secret Access Key: <same>
# Default region: ap-south-1
# Default output format: json

aws sts get-caller-identity # confirm ARN shows haru-devops, not root
```

⚠ Common Issues — Account Setup

Issue	Cause	Fix
Access keys accidentally exposed (e.g. pasted in chat/repo)	Credentials leaked outside a secure vault	Rotate/delete the key immediately in IAM → Security Credentials; never store keys in code or chat
aws sts get-caller-identity shows root ARN	CLI still configured with root keys	Re-run aws configure with the IAM user's keys
Billing alarm never triggers	Billing alerts not enabled in Billing Preferences first	Enable 'Receive Billing Alerts' before creating the CloudWatch alarm

3. Infrastructure as Code — Terraform

All infrastructure below is provisioned via Terraform for reproducibility. Folder structure:

```
hr-system-infra/  
└─ terraform/  
   ├── main.tf  
   ├── variables.tf  
   ├── vpc.tf  
   ├── eks.tf  
   ├── ecr.tf  
   ├── rds.tf  
   ├── iam.tf  
   └─ outputs.tf
```

3.1 main.tf — Provider & Backend

```
terraform {  
  required_version = ">= 1.6"  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
  backend "s3" {  
    bucket = "hr-system-tfstate"  
    key    = "infra/terraform.tfstate"  
    region = "ap-south-1"  
  }  
}  
  
provider "aws" {  
  region = var.aws_region  
}
```

3.2 vpc.tf — VPC, Subnets, NAT (Multi-AZ)

```
module "vpc" {  
  source = "terraform-aws-modules/vpc/aws"  
  version = "~> 5.0"  
  
  name = "hr-system-vpc"  
  cidr = "10.0.0.0/16"  
  
  azs          = ["ap-south-1a", "ap-south-1b", "ap-south-1c"]  
  public_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]  
  private_subnets = ["10.0.11.0/24", "10.0.12.0/24", "10.0.13.0/24"]  
  database_subnets = ["10.0.21.0/24", "10.0.22.0/24", "10.0.23.0/24"]  
  
  enable_nat_gateway = true  
  single_nat_gateway = true # set false + one_nat_gateway_per_az = true for production HA  
  
  create_database_subnet_group = true  
  
  tags = {  
    Project = "hr-management-system"  
  }  
}
```

3.3 eks.tf — EKS Cluster (multi-AZ worker nodes)

```
module "eks" {
  source = "terraform-aws-modules/eks/aws"
  version = "~> 20.0"

  cluster_name      = "hr-eks-cluster"
  cluster_version   = "1.29"

  vpc_id            = module.vpc.vpc_id
  subnet_ids        = module.vpc.private_subnets

  enable_irsa       = true

  eks_managed_node_groups = {
    default = {
      instance_types = ["t3.medium"]
      min_size        = 2
      max_size        = 4
      desired_size    = 2
      subnet_ids      = module.vpc.private_subnets # spans all 3 AZs
    }
  }

  tags = {
    Project = "hr-management-system"
  }
}
```

3.4 ecr.tf — Container Registries

```
resource "aws_ecr_repository" "backend" {
  name                  = "hr-backend"
  image_tag_mutability = "IMMUTABLE"
  image_scanning_configuration { scan_on_push = true }
}

resource "aws_ecr_repository" "frontend" {
  name                  = "hr-frontend"
  image_tag_mutability = "IMMUTABLE"
  image_scanning_configuration { scan_on_push = true }
}
```

3.5 rds.tf — PostgreSQL (isolated DB subnet)

```
resource "aws_db_subnet_group" "hr_db" {
  name      = "hr-db-subnet-group"
  subnet_ids = module.vpc.database_subnets
}

resource "aws_security_group" "rds_sg" {
  name      = "hr-rds-sg"
  vpc_id    = module.vpc.vpc_id

  ingress {
    from_port = 5432
    to_port   = 5432
    protocol  = "tcp"
    security_groups = [module.eks.node_security_group_id]
  }
}
```

```

resource "aws_db_instance" "hr_postgres" {
  identifier      = "hr-db-server"
  engine          = "postgres"
  engine_version  = "15"
  instance_class  = "db.t3.micro"
  allocated_storage = 32
  db_name         = "hrdb"
  username        = "hradmin"
  password        = var.db_password # supply via TF_VAR or secrets, never hardcode
  db_subnet_group_name = aws_db_subnet_group.hr_db.name
  vpc_security_group_ids = [aws_security_group.rds_sg.id]
  multi_az        = false # set true for production HA
  skip_final_snapshot = true
}

```

3.6 iam.tf — IRSA Role for Backend (Secrets Manager access)

```

module "irsa_backend" {
  source = "terraform-aws-modules/iam/aws//modules/iam-role-for-service-accounts-eks"
  version = "~> 5.0"

  role_name = "hr-backend-irsa"

  oidc_providers = {
    main = {
      provider_arn          = module.eks.oidc_provider_arn
      namespace_service_accounts = ["default:hr-backend-sa"]
    }
  }
}

resource "aws_iam_role_policy" "backend_secrets_access" {
  name = "hr-backend-secrets-access"
  role = module.irsa_backend.iam_role_name

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = ["secretsmanager:GetSecretValue"]
      Resource = "arn:aws:secretsmanager:ap-south-1*:secret:hr-system/*"
    }]
  })
}

```

3.7 Apply

```

terraform init
terraform plan
terraform apply

aws eks update-kubeconfig --name hr-eks-cluster --region ap-south-1
kubectl get nodes

```

[⚠ Common Issues — Terraform / Infra](#)

Issue	Cause	Fix
terraform apply fails: bucket already exists	S3 backend bucket name not globally unique	Choose a unique bucket name for tfstate storage
EKS nodes stuck NotReady	Node IAM role missing required policies	Ensure AmazonEKSWorkerNodePolicy and CNI policy attached to node role
RDS unreachable from pods	Security group not allowing EKS node SG on port 5432	Verify aws_security_group ingress rule references EKS node security group
kubectrl get nodes: Unauthorized	kubeconfig not updated / wrong IAM identity	Re-run aws eks update-kubeconfig; confirm aws sts get-caller-identity matches cluster access

4. Containerization

4.1 Backend Dockerfile (Spring Boot)

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

4.2 Frontend Dockerfile (React)

```
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json .
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
```

4.3 Build, Scan & Push (manual reference — Jenkins automates this)

```
aws ecr get-login-password --region ap-south-1 | \
  docker login --username AWS --password-stdin <account-id>.dkr.ecr.ap-south-1.amazonaws.com

docker build -t hr-backend:v1 .
docker tag hr-backend:v1 <account-id>.dkr.ecr.ap-south-1.amazonaws.com/hr-backend:v1
trivy image hr-backend:v1 --severity HIGH,CRITICAL
docker push <account-id>.dkr.ecr.ap-south-1.amazonaws.com/hr-backend:v1
```

⚠ Common Issues — Containerization

Issue	Cause	Fix
docker push: no basic auth credentials	ECR login expired	Re-run aws ecr get-login-password and docker login
Trivy blocks the build	Base image has known HIGH/CRITICAL CVEs	Update to a newer slim/alpine base image tag
Image too large	Full JDK/node image used instead of slim variant	Use multi-stage builds as shown above

5. Jenkins CI/CD Pipeline

5.1 Jenkins Master on EKS

Jenkins master runs as a pod in the private subnet with an EBS-backed PersistentVolumeClaim so plugin/job configuration survives pod restarts.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: jenkins-pvc
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: gp3
  resources:
    requests:
      storage: 10Gi
```

5.2 Jenkinsfile — Full Pipeline

```
pipeline {
  agent any
  environment {
    ECR_REGISTRY = "<account-id>.dkr.ecr.ap-south-1.amazonaws.com"
    AWS_REGION   = "ap-south-1"
  }
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Backend Build & Test') {
      steps {
        dir('backend') { sh 'mvn clean package' }
      }
    }
    stage('Frontend Build') {
      steps {
        dir('frontend') { sh 'npm install && npm run build' }
      }
    }
    stage('SonarQube Quality Gate') {
      steps { sh 'mvn sonar:sonar' }
    }
    stage('Docker Build') {
      steps {
        sh 'docker build -t $ECR_REGISTRY/hr-backend:$BUILD_NUMBER backend/'
        sh 'docker build -t $ECR_REGISTRY/hr-frontend:$BUILD_NUMBER frontend/'
      }
    }
    stage('Trivy Scan') {
      steps {
        sh 'trivy image $ECR_REGISTRY/hr-backend:$BUILD_NUMBER --severity HIGH,CRITICAL'
        sh 'trivy image $ECR_REGISTRY/hr-frontend:$BUILD_NUMBER --severity HIGH,CRITICAL'
      }
    }
    stage('Push to ECR') {
      steps {
        sh 'aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin $ECR_REGISTRY'
        sh 'docker push $ECR_REGISTRY/hr-backend:$BUILD_NUMBER'
        sh 'docker push $ECR_REGISTRY/hr-frontend:$BUILD_NUMBER'
      }
    }
  }
}
```

```

stage('Deploy to EKS (Helm)') {
    steps {
        sh 'helm upgrade --install hr-backend ./charts/backend --set image.tag=$BUILD_NUMBER'
        sh 'helm upgrade --install hr-frontend ./charts/frontend --set image.tag=$BUILD_NUMBER'
    }
}
}
}

```

⚠ Common Issues — Jenkins / CI-CD

Issue	Cause	Fix
Jenkins can't push to ECR	Jenkins pod's service account lacks ECR permissions	Attach IRSA role with AmazonEC2ContainerRegistryPowerUser to the Jenkins service account
helm upgrade fails: connection refused	Jenkins pod lacks cluster access	Ensure Jenkins service account has a ClusterRoleBinding for kubectl/helm access
Old image still running after deploy	Reused latest tag instead of unique BUILD_NUMBER	Always tag images with \$BUILD_NUMBER or git SHA

6. Kubernetes Deployment Manifests

6.1 Backend Deployment + Service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hr-backend
spec:
  replicas: 2
  selector:
    matchLabels: { app: hr-backend }
  template:
    metadata:
      labels: { app: hr-backend }
    spec:
      serviceAccountName: hr-backend-sa
      containers:
        - name: hr-backend
          image: <account-id>.dkr.ecr.ap-south-1.amazonaws.com/hr-backend:latest
          ports: [{ containerPort: 8080 }]
          envFrom:
            - secretRef: { name: hr-backend-secrets }
            - configMapRef: { name: hr-backend-config }
          resources:
            requests: { cpu: "250m", memory: "512Mi" }
            limits: { cpu: "500m", memory: "1Gi" }
          readinessProbe:
            httpGet: { path: /actuator/health, port: 8080 }
            initialDelaySeconds: 20
          livenessProbe:
            httpGet: { path: /actuator/health, port: 8080 }
            initialDelaySeconds: 30
---
apiVersion: v1
kind: Service
metadata:
  name: hr-backend-service
spec:
  selector: { app: hr-backend }
  ports: [{ port: 8080, targetPort: 8080 }]
  type: ClusterIP
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: hr-backend-sa
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::<account-id>:role/hr-backend-irsa
```

6.2 Frontend Deployment + Service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hr-frontend
spec:
  replicas: 2
  selector:
    matchLabels: { app: hr-frontend }
  template:
    metadata:
      labels: { app: hr-frontend }
    spec:
```

```

    containers:
      - name: hr-frontend
        image: <account-id>.dkr.ecr.ap-south-1.amazonaws.com/hr-frontend:latest
        ports: [{ containerPort: 80 }]
        envFrom:
          - configMapRef: { name: hr-frontend-config }    # non-sensitive only
---
apiVersion: v1
kind: Service
metadata:
  name: hr-frontend-service
spec:
  selector: { app: hr-frontend }
  ports: [{ port: 80, targetPort: 80 }]
  type: ClusterIP

```

6.3 HPA — Autoscaling for Both Services

```

kubectl autoscale deployment hr-backend --cpu-percent=70 --min=2 --max=6
kubectl autoscale deployment hr-frontend --cpu-percent=70 --min=2 --max=4

```

6.4 Nginx Ingress — Path-Based Routing

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: hr-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service: { name: hr-backend-service, port: { number: 8080 } }
          - path: /
            pathType: Prefix
            backend:
              service: { name: hr-frontend-service, port: { number: 80 } }

```

```

helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm install nginx-ingress ingress-nginx/ingress-nginx
kubectl apply -f ingress.yaml
kubectl get ingress

```

⚠ Common Issues — Kubernetes / Ingress

Issue	Cause	Fix
Pods Pending	Insufficient node capacity	<code>kubectl describe pod <name></code> ; scale node group min/max
ImagePullBackOff	Node IAM role missing ECR pull permission	Attach <code>AmazonEC2ContainerRegistryReadOnly</code> to node IAM role
CrashLoopBackOff on backend	Missing secrets/config or DB unreachable	<code>kubectl logs <pod></code> ; check hr-backend-secrets values and RDS security group

Issue	Cause	Fix
Ingress ADDRESS stays <pending>	AWS Load Balancer Controller not installed	Install the AWS Load Balancer Controller add-on via Helm before creating Ingress
404 on /api/* routes	Ingress path/rewrite misconfigured	Verify rewrite-target annotation and Spring Boot base path

7. Secrets Management — Secrets Manager + IRSA + ESO

7.1 Store Secrets in AWS Secrets Manager

```
aws secretsmanager create-secret \  
  --name hr-system/db-credentials \  
  --secret-string '{"username":"hradmin","password":"<StrongPassword>","url":"jdbc:postgresql://hr-db-  
server.xxxx.ap-south-1.rds.amazonaws.com:5432/hrdb"}'  
  
aws secretsmanager create-secret \  
  --name hr-system/jwt-secret \  
  --secret-string '{"jwtSecret":"<your-jwt-signing-key>"}'
```

7.2 Install External Secrets Operator (ESO)

```
helm repo add external-secrets https://charts.external-secrets.io  
helm install external-secrets external-secrets/external-secrets -n external-secrets --create-namespace
```

7.3 SecretStore + ExternalSecret (backend only)

```
apiVersion: external-secrets.io/v1beta1  
kind: SecretStore  
metadata:  
  name: aws-secrets-store  
spec:  
  provider:  
    aws:  
      service: SecretsManager  
      region: ap-south-1  
      auth:  
        jwt:  
          serviceAccountRef:  
            name: hr-backend-sa  
---  
apiVersion: external-secrets.io/v1beta1  
kind: ExternalSecret  
metadata:  
  name: hr-backend-secrets  
spec:  
  refreshInterval: 1h  
  secretStoreRef:  
    name: aws-secrets-store  
    kind: SecretStore  
  target:  
    name: hr-backend-secrets  
  data:  
  - secretKey: DB_URL  
    remoteRef: { key: hr-system/db-credentials, property: url }  
  - secretKey: DB_USER  
    remoteRef: { key: hr-system/db-credentials, property: username }  
  - secretKey: DB_PASSWORD  
    remoteRef: { key: hr-system/db-credentials, property: password }  
  - secretKey: JWT_SECRET  
    remoteRef: { key: hr-system/secret, property: jwtSecret }
```

7.4 Non-Sensitive Config — ConfigMap (frontend + backend)

```
apiVersion: v1  
kind: ConfigMap  
metadata:
```



```
name: hr-frontend-config
data:
  API_BASE_URL: "/api"
  LOG_LEVEL: "info"
```

Common Issues — Secrets

Issue	Cause	Fix
ExternalSecret stuck SecretSyncError	IRSA role not authorized for secretsmanager:GetSecretValue	Verify IAM policy resource ARN matches the secret name/prefix
Pod can't read hr-backend- secrets	ExternalSecret target name mismatch with envFrom.secretRef	Ensure names match exactly between ExternalSecret target and Deployment envFrom
Frontend pod has DB credentials	Secrets accidentally wired into frontend Deployment	Frontend should only use ConfigMap — remove any secretRef from frontend manifest

8. Monitoring — Prometheus, CloudWatch, Grafana

8.1 Expose Spring Boot Metrics

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>

management:
  endpoints:
    web:
      exposure:
        include: health,prometheus,metrics
```

8.2 Amazon Managed Service for Prometheus (AMP)

```
aws amp create-workspace --alias hr-system-amp

# Install the ADOT collector to scrape pods and remote-write to AMP
helm repo add aws-observability https://aws-observability.github.io/aws-otel-helm-charts
helm install adot-collector aws-observability/adot-exporter-for-eks-on-ec2 \
  --set awsRegion=ap-south-1 \
  --set clusterName=hr-eks-cluster
```

8.3 AWS Managed Grafana

- Create a Managed Grafana workspace in the AWS console.
- Add AMP as a data source (IAM role-based auth, no manual credentials).
- Import or build dashboards for CPU/memory usage, API latency, and DB connection pool health.

8.4 Alerts

- Pod restarts / CrashLoopBackOff
- High CPU/memory (>80%)
- API 5xx error rate spike
- Database connection pool exhaustion

Common Issues — Monitoring

Issue	Cause	Fix
No data in Grafana	Nothing deployed yet, or ADOT collector not scraping	Confirm pods are running first; check ADOT collector logs
ADOT can't write to AMP	IRSA role for ADOT missing aps:RemoteWrite permission	Attach an IAM policy granting aps:RemoteWrite to the ADOT service account
Grafana workspace can't query AMP	Data source auth not configured for IAM SigV4	Use the AWS SigV4 data source type when adding AMP in Grafana

9. Security & Compliance Checklist

- Root account: MFA enabled, no access keys, used only for billing/account recovery.
- IAM user: MFA enabled, used for all daily operations.
- Secrets never hardcoded — stored in Secrets Manager, synced via ESO.
- Backend-only secret access — frontend pods never receive DB credentials or tokens.
- RDS in isolated subnet, security group scoped to EKS node SG only.
- Trivy scanning on every image build; pipeline fails on HIGH/CRITICAL CVEs.
- SonarQube quality gate before merge/deploy.
- SSL/TLS via ALB + ACM certificate (or cert-manager if using Nginx directly).
- CloudTrail enabled for full account audit history.

10. Cost Optimization

Area	Optimization
EKS Nodes	t3.medium with cluster autoscaler (min 2, max 4); consider Spot for non-prod
NAT Gateway	Single NAT for dev/staging; one-per-AZ only for production
RDS	db.t3.micro, Multi-AZ off for dev; enable Multi-AZ only for production
ECR	Lifecycle policy to expire untagged/old images
Monitoring	Reasonable AMP retention; sample high-cardinality metrics
Governance	Billing alarms + AWS Budgets already configured (Section 2.3)

11. Deployment Checklist

- ☐ Root account MFA enabled, no root access keys
- ☐ IAM user created with MFA, billing alarms and budgets configured
- ☐ Terraform: VPC (3 AZs), EKS cluster, ECR repos, RDS, IRSA roles provisioned
- ☐ kubectl get nodes returns healthy multi-AZ nodes
- ☐ Backend + frontend Dockerized, images built and scanned
- ☐ Jenkins pipeline live: build → test → SonarQube → Trivy → ECR → Helm deploy
- ☐ Kubernetes manifests deployed (Deployments, Services, ServiceAccount)
- ☐ HPA configured for backend and frontend
- ☐ Nginx Ingress + AWS Load Balancer Controller routing traffic correctly
- ☐ Secrets Manager + ESO syncing secrets to backend pods only
- ☐ ConfigMap serving non-sensitive config to both services
- ☐ AMP + Managed Grafana showing live metrics
- ☐ Alerts configured and tested (crash loop, latency, error rate)
- ☐ Backend confirmed calling Microsoft Graph API (not frontend)

Document prepared as internship DevOps reference — full end-to-end AWS/EKS deployment for the HR Management System, from account setup through monitoring.